October 20 Practice: Bang!

Keeley Dalton

2026-10-20

Setting up workspace.

```
library(readr)
library(dplyr)
library(stringr)
library(tidyverse)
library(tinytable)
```

The first step to obtaining and looking into Upworthy's A/B testing data, which they use to optimize the content on their website, is to read the csv file into R.

```
upworthy <- read_csv("https://osf.io/vy8mj/download")
```

```
New names:
Rows: 105551 Columns: 17
-- Column specification
-------------------------------------------------------- Delimiter: "," chr
(8): clickability_test_id, excerpt, headline, lede, slug, eyecatcher_id... dbl
(5): ...1, impressions, clicks, significance, test_week lgl (2): first_place,
winner dttm (2): created_at, updated_at
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `` -> `...1`
```

Now that this data is read in, I can pull out the names.

```
upworthy |>
  names()
```

```
 [1] "...1"                 "created_at"           "updated_at"
 [4] "clickability_test_id" "excerpt"              "headline"
 [7] "lede"                 "slug"                 "eyecatcher_id"
[10] "impressions"          "clicks"               "significance"
[13] "first_place"          "winner"               "share_text"
[16] "square"               "test_week"
```

Then, I can pull out the top six results from the data.

```
upworthy |>
  head()
```

```
# A tibble: 6 x 17
   ...1 created_at          updated_at          clickability_test_id     excerpt
  <dbl> <dttm>              <dttm>              <chr>                    <chr>
1    11 2014-11-20 11:33:26 2016-04-02 16:25:54 546dd17e26714c82cc00001c Things~
2    12 2014-11-20 15:00:01 2016-04-02 16:25:54 546e01d626714c6c4400004e Things~
3    13 2014-11-20 11:33:51 2016-04-02 16:25:54 546dd17e26714c82cc00001c Things~
4    14 2014-11-20 11:34:12 2016-04-02 16:25:54 546dd17e26714c82cc00001c Things~
5    15 2014-11-20 11:34:33 2016-04-02 16:25:54 546dd17e26714c82cc00001c Things~
6    16 2014-11-20 11:34:48 2016-04-02 16:25:54 546dd17e26714c82cc00001c Things~
# i 12 more variables: headline <chr>, lede <chr>, slug <chr>,
#   eyecatcher_id <chr>, impressions <dbl>, clicks <dbl>, significance <dbl>,
#   first_place <lgl>, winner <lgl>, share_text <chr>, square <chr>,
#   test_week <dbl>
```

I am following Alexander in the textbook, but using a different selection criteria ("!" instead of "?") so I am choosing the columns "created_at," "clickability_test_id," "headline," "impressions," and "clicks." The following code chunk makes a new data frame for this restricted data and pulls those columns out and then uses head to show the top six responses to this data.

```
upworthy_restricted <-
  upworthy |>
  select(
    created_at, clickability_test_id, headline, impressions, clicks
    )

head(upworthy_restricted)
```

```
# A tibble: 6 x 5
  created_at          clickability_test_id     headline       impressions clicks
  <dttm>              <chr>                    <chr>                <dbl>  <dbl>
1 2014-11-20 11:33:26 546dd17e26714c82cc00001c Let's See … H~        3118      8
2 2014-11-20 15:00:01 546e01d626714c6c4400004e People Sent T~        4587    130
3 2014-11-20 11:33:51 546dd17e26714c82cc00001c $3 Million Is~        3017     19
4 2014-11-20 11:34:12 546dd17e26714c82cc00001c The Fact That~        2974     26
5 2014-11-20 11:34:33 546dd17e26714c82cc00001c Reason #351 T~        3050     10
6 2014-11-20 11:34:48 546dd17e26714c82cc00001c I Was Already~        3061     20
```

Here is where my code differs from the textbook. Instead of using "asks_questions," I am using "uses_bang." We're looking at the text in headlines, and specifically those headlines that used "!" This chunk is telling R to bring up the data for those headlines that used a "!" and used the same image, in order to remove the effect of the image differences themselves.

```
upworthy_restricted <-
  upworthy_restricted |>
  mutate(
    uses_bang =
      str_detect(string = headline, pattern = "\\!")
    )

upworthy_restricted |>
  count(uses_bang)
```

```
# A tibble: 2 x 2
  uses_bang      n
  <lgl>      <int>
1 FALSE     103514
2 TRUE        2037
```

Now, for each text with "!" and each picture, I want to see whether the bang has an effect on the number of clicks.

```
uses_bang_or_not <-
  upworthy_restricted |>
  summarise(
    ave_clicks = mean(clicks),
    .by = c(clickability_test_id, uses_bang)
  )

uses_bang_or_not |>
  pivot_wider(names_from = uses_bang,
              values_from = ave_clicks,
              names_prefix = "ave_clicks_") |>
  drop_na(ave_clicks_FALSE, ave_clicks_TRUE) |>
  mutate(difference_in_clicks = ave_clicks_TRUE - ave_clicks_FALSE) |>
  summarise(average_differce = mean(difference_in_clicks))
```

```
# A tibble: 1 x 1
  average_differce
             <dbl>
1            -2.93
```

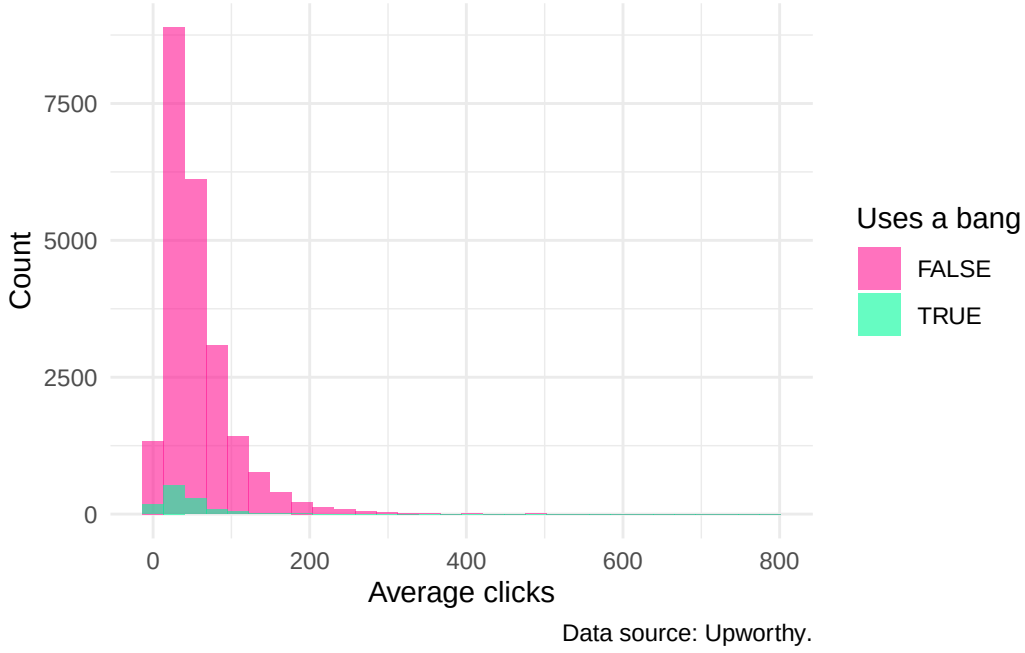
This code will provide a cross tab showing the difference between the average number of click between bang and no bang.

```
uses_bang_or_not |>
  summarise(mean = mean(ave_clicks),
            .by = uses_bang) |>
  tt() |>
  style_tt(j = 1:2, align = "lr") |>
  format_tt(digits = 0, num_fmt = "decimal") |>
  setNames(c("UsesABang!", "MeanClicks"))
```

UsesABang!	MeanClicks
FALSE	57
TRUE	42

This is my histogram.

```
uses_bang_or_not |>
  ggplot(aes(x = ave_clicks, fill = factor(uses_bang))) +
  geom_histogram(position = "identity", alpha = 0.6, bins = 30) +
  scale_fill_manual(values = c("deeppink", "mediumspringgreen")) +
  theme_minimal() +
  labs(
    x = "Average clicks",
    y = "Count",
    fill = "Uses a bang",
    caption = "Data source: Upworthy."
  )
```



Yay! The end.